

REASONGUARD: Verifiable Guardrails for Natural-Language Explanations from Formal Anomaly Detectors in Safety-Critical Systems

Ravan Bilalov

M.Sc. Applied Data Science and Analytics

SRH University Hamburg, Germany

in collaboration with the Faculty of Information Technology,
Brno University of Technology (NES@FIT), Czech Republic

Ravan.Bilalov@stud.srh-university.de

Abstract—Large Language Models (LLMs) are increasingly used to turn machine-readable outputs from formal anomaly detectors into natural-language explanations. These explanations can be hallucinated or otherwise unfaithful to the detector’s actual verdict, potentially misleading human operators. We propose REASONGUARD, a runtime verifier that enforces machine-checkable faithfulness criteria on explanations of detector verdicts, defined over a specification of the monitored property. REASONGUARD is deterministic, auditable, and fail-closed, enabling safe deployment of LLM-generated explanations in safety-critical settings. We apply REASONGUARD in the context of anomaly detection on private network tele-control traffic from an electrical substation conforming to the IEC-104 protocol. We present a taxonomy of faithfulness violations, provide a formalization of each violation class along with the corresponding violation-detection rule, and give actionable explanations of violations.

We demonstrate REASONGUARD’s applicability on a corpus of 500 LLM-generated explanations of detector verdicts on smart-grid network traffic from Brno University of Technology, comprising outputs from three locally-run LLMs and two cloud-based LLMs, and 500 synthetic explanations crafted to induce known violations as regression test cases. We provide a benchmark of 200 ground-truth annotations produced by human annotators. REASONGUARD flags violations with varying reliability: for example, it detects fabricated entities with precision of 0.90 but only recall of 0.09, and under-specification with precision of 0.48 and recall of 0.92. We find statistically significant associations between violation occurrence and the LLM used to generate explanations as well as the type of event being explained. REASONGUARD has limited agreement with human-generated annotations ($\kappa = 0.16$), primarily due to overly permissive criteria resulting in overtriggering, indicating its applicability in human-in-the-loop settings. REASONGUARD’s fine-grained violation classes (five in total, referred to as V1–V5), actionable explanations of violations, and fail-closed design enable human-in-the-loop use-cases by prioritizing explanations for human review, and map to Article 13 (transparency) and Article 17 (quality management) of the EU AI Act. We release our implementation, prompts, and datasets under an open-source license.

Index Terms—runtime verification, faithfulness, large language models, formal methods, industrial control systems, IEC-104, EU AI Act, explainability

I. INTRODUCTION

Large language models (LLMs) can be used to generate natural-language explanations of data or predictions, making them attractive in high-stakes domains where decisions made on the basis of that data need to be auditable and comprehensible to domain experts. One such domain is cybersecurity. Existing work on runtime verification for industrial control systems uses finite-state automata that formally model the specification of correct behavior in private tele-control networks connecting substations in a smart-grid. These monitors observe traffic conforming to the IEC-104 protocol in real time, and raise an alarm with a certificate, that is, a proof, of the specification violation that can indicate a network attack. Operators can be alerted about attacks by viewing the verdicts of such anomaly-detection programs. However, the verdicts themselves can be difficult to understand, making it challenging for operators to act upon them in a timely fashion. Recent work has explored using LLMs to generate natural-language explanations of verdicts raised by such programs. In critical use-cases such as monitoring for network attacks, it is paramount that these explanations are faithful to the verdicts raised by these programs, as false positives could result in wasted effort and false negatives could result in real attacks going unnoticed.

Standard methods of evaluating LLM-generated explanations operate offline on a set of inputs [1], [2], [3]. In contrast, in order to safely deploy such explanations we seek guarantees about the faithfulness of any individual explanation generated in response to any individual input from the anomaly detector at runtime. This motivates the use of runtime verification to enforce criteria that any LLM-generated explanation must satisfy. We propose REASONGUARD, a framework that uses runtime verification to provide machine-checkable guarantees about the faithfulness of LLM-generated natural-language explanations of verdicts from anomaly-detection programs. REASONGUARD takes a specification of the property being

monitored by the anomaly-detection program along with the verdict raised by the program on the monitored system, and verifies whether the LLM-generated explanation adheres to criteria for being faithful to the verdict. REASONGUARD raises alarms in case of unfaithfulness with actionable explanations.

Contributions. First, we propose REASONGUARD, a novel, deterministic, auditable, and fail-closed verifier that checks whether LLM-generated, natural-language explanations of formal verification results are faithful to the results. We instantiate it for checking explanations of outputs from an anomaly detector for the IEC-104 industrial control system communication protocol (section III).

Second, we conduct an extensive empirical study of REASONGUARD, including:

- a dataset of 500 LLM-generated explanations of IEC-104 detector verdicts using three local and two cloud-based LLMs;
- a suite of 500 synthetic test cases for regression-testing REASONGUARD and a benchmark of 200 blind ground-truth annotations;
- evidence for a statistically significant association between the faithfulness violations flagged by REASONGUARD and both the model that produced the explanation and the event being explained;
- a discussion of false alarms, and of using REASONGUARD in the loop with a human analyst as a triage tool;
- a mapping of REASONGUARD to Articles 13 and 17 of the EU AI Act [4];
- open-source, fully reproducible research artifacts [5].

Third, we introduce a taxonomy of types of unfaithfulness in LLM-generated natural-language explanations of anomaly-detection results, along with rules to detect these types and an indication of their severity for prioritizing analyst attention.

II. BACKGROUND AND RELATED WORK

A. Formal methods and automaton-based ICS detection

Model checking is a formal verification technique that determines whether a given model of a finite-state system fulfills some specification formulated in some temporal logic [6]. In the context of ICSs, the specification describes the system’s legitimate behavior. Observing a violation of the specification thus gives a definite indication that an anomaly has occurred with respect to the system’s model. Havlena et al. [7] infer a deterministic probabilistic automaton using legitimate IEC-104 and MMS traffic. They also take into account the semantics of the messages, reducing false positives. The resulting anomaly detector produces a structured verdict describing the nature of the anomaly, which devices were involved, when it started, etc. This verdict is used by REASONGUARD as an inviolable specification for its explanations.

Due to the sensitivity of LLMs to prompts and their opaque internals, it is not feasible to bound and model them like other programs and prove that their executions satisfy some property. This is where runtime verification (RV) comes in,

where, instead of modeling the LLM and verifying all its executions, we verify a single execution of it.

B. Faithfulness evaluation

Faithfulness evaluation can be separated into branches: world factuality that measures whether the generation aligns with world knowledge; source faithfulness that measures whether the generation is entailed by the source [1]. FACTSCORE [2] decomposes a given output into atomic facts and validates them with a knowledge base. TRUE [3] integrates 11 benchmarks for faithfulness evaluation and shows that natural language inference measures are complementary to question generation measures. Similar to them, REASONGUARD decomposes a given output into atomic claims.

Differently, we provide a deterministic and auditable process suitable for certification and propose an actionable class instead of a scalar value. Moreover, our work focuses on faithfulness to verdicts provided by formal methods, which differs from faithfulness to source documents and has received little attention.

C. Runtime verification

Runtime verification determines whether a single run of a system obeys some specification, all during execution [8]. The monitor is usually generated from a specification. In our setting, the execution is the explanation, the specification is our bound and the monitor is the verifier. This correspondence is what allows us to apply runtime verification in REASONGUARD.

Faithfulness is a safety property since a finite input reveals whether there is a violation: a single incorrect assertion violates the specification, independent of the number of correct ones. Our bound is fail-closed since any assertion not explicitly allowed is considered incorrect. This ensures that the monitor is independent of the generator. We contribute a novel application of RV and a novel RV monitor.

D. The EU AI Act

Under the AI Act, Regulation (EU) 2024/1689 [4], AI systems that are a safety component in the management and operation of essential infrastructures in the areas of electricity, gas, and water fall under high-risk systems according to Annex III. For high-risk AI systems, providers should ensure that the systems are transparent enough for the recipients to be able to interpret and use the output appropriately under Article 13. While REASONGUARD is not a control component, it can help to provide evidence for fulfilling this requirement.

Furthermore, providers are expected to implement a quality-management framework under Article 17 that entails appropriate measures to monitor, document, test, and take corrective actions. REASONGUARD can help provide evidence for such a process by monitoring the faithfulness of the explanation shown to the operator, see section VI for more details.

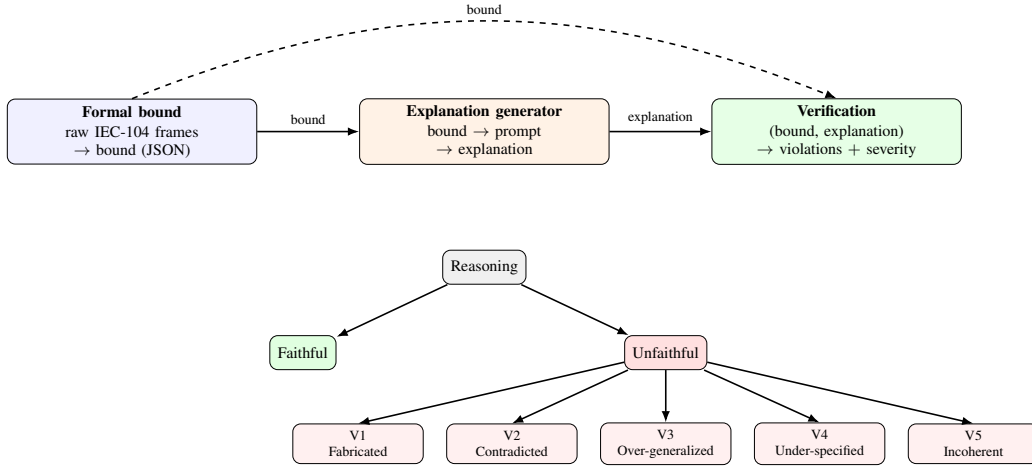


Fig. 1. The REASONGUARD architecture (top) and the reasoning-violation taxonomy (bottom). The formal bound is built from raw frames; the explanation generator turns the bound into an LLM explanation; verification checks the explanation against the bound and reports violations with a severity. Reasoning is either faithful or unfaithful, and unfaithful reasoning is partitioned into the five classes V1–V5.

TABLE I
SEVERITY MAPPING OF THE VIOLATION TAXONOMY.

Violation	Meaning	Severity
V1	Fabricated	High
V2	Contradicted	High
V3	Over-generalized	Low
V4	Under-specified	Low
V3 $\wedge$ V4	Both omission modes	Medium
V5	Incoherent	High

III. THE REASONGUARD FRAMEWORK

A. Reasoning as assertions

We operationalize reasoning as an explanation containing assertions about causes and consequences. To evaluate reasoning we identify reasoning violations, meaning claims that do not follow from the explanation’s input. We propose a taxonomy of violations (fig. 1) and a way to detect them automatically. Our taxonomy splits unfaithful reasoning (fig. 1, bottom) into five violation types (table I):

- **V1 Fabricated:** a fact or entity without evidence.
- **V2 Contradicted:** the opposite of a fact.
- **V3 Over-generalized:** the correct category without specific details.
- **V4 Under-specified:** an assertion that should be made is omitted.
- **V5 Incoherent:** two statements contradict each other.

A fabricated entity is high severity as it may prompt the deployer to consider unrelated traffic. Contradicting “normal” behavior may hide attacks; inventing “malicious” behavior may cause disruption. Hence, V1, V2 and V5 are high severity. Missing details or assertions, V3 and V4, are low severity in isolation, medium severity if both types are present.

B. Architectural overview

REASONGUARD is loosely coupled with the aforementioned methods, meaning that other techniques can be used instead.

C. Formal bound

The formal bound is central to this work and the core of our proposed formalization. It is a JSON object with several fields (listing 1). Besides meta-information such as the detector’s name, it stores observable features of an input, a list of allowed, required, and forbidden claims, that is, concepts and relations, and several Booleans encoding machine-constraints. Essentially, this object can be seen as a detector’s verdict. Since the schema used by Havlena et al.’s detector [7] and the automata maintained by the official NES@FIT page were unavailable, we constructed a proxy bound using the labels from the dataset. As a result, our implementation generates a bound that is fail-closed, meaning all machine-constraint fields are false and the bound receives a proxy label. Moreover, the rules explicitly warn deployers not to interpret such an explanation as approval.

D. Explanation generator

The generator is a black-box large language model. Our implementation allows for seamless switching between local models and LLM APIs. Its input is the prompt, which contains the same observable features that are used for generating a formal bound.

E. Verification

Given a formal bound and an AI-generated explanation, REASONGUARD verifies the explanation. The resulting verdict is an empty list in case of a consistent explanation; otherwise, a list of violations. As an illustration, listing 1 shows a canonical formal bound created from raw IEC-104 protocol traffic.

IV. EXPERIMENTAL SETUP

Dataset and protocol coverage. We use the publicly available *ICS Dataset for Smart Grid Anomaly Detection* [9] from Brno University of Technology. For this paper, we report results

Listing 1. Canonical formal bound (nproxv schema v2.1).

```

{
  "event_id": "evt_000001",
  "dataset": "eon_iec",
  "formal_bound_version": "2.1",
  "schema_status": "proxy_until_official_automaton_schema",
  "protocol_hint": "iec104",
  "communication_direction": "client_to_server",
  "formal_class": "IEC104_CONTROL_FRAME_OBSERVATION",
  "features": {
    "source_id": 1, "destination_id": 2,
    "srcport": 57528, "dstport": 2404, "
    asdu_type": 0 },
  "allowed_claims": [ "... 10 atomic statements ..." ],
  "required_claims": [ "... 5 must-mention statements ..." ],
  "forbidden_claims": [ "... 8 must-not-mention statements ..." ],
  "machine_constraints": { "attack": false, "malicious": false,
    "mitigation": false, "causal": false, "normality": false },
  "proxy_event_type": "
    topology_change_proxy_until_official_schema"
}

```

for the `eon-iec` subset, a collection of 20,000 atomic IEC-104 protocol records, where each row represents one IEC-104 frame. Each row is transformed into a bound using the procedure described in section III. Replay-attack anomalies are present only in the Modbus part of the dataset [10] and are therefore omitted here; the remaining four event types are denoted as follows:

- **E1:** normal polling, representing a normal (non-anomalous) event;
- **E2:** function-code violation, representing a violation of IEC-104 function-code usage rules;
- **E3:** network scan, representing port-scanning behavior;
- **E4:** topology change, representing a change of communication endpoints.

Models and inference settings. We evaluate three local models, specifically `llama3.1:8b`, `mistral:7b`, and `phi3:mini`, via Ollama with 4-bit quantization on CPU, as well as two proprietary models, namely `gpt-4o-mini` and `gemini-2.5-flash-lite`, accessed through their respective APIs. To reduce variance, all models are configured with temperature 0.1 and top- p 0.9. For each model we generate explanations on 100 prompts sampled evenly from the dataset, with 25 per event type (E1–E4). We use a common, detailed (verbose) system prompt for all models, encouraging verbose answers while forbidding unsupported or speculative claims. Hosts are not included in the prompt, since they appear as anonymized/obfuscated identifiers.

Claim extraction. We use spaCy for part-of-speech (POS) tagging and named-entity recognition (NER). We implement a custom spaCy `EntityRuler` to perform NER for the closed set of claim categories first, and run statistical tagging for the remaining (open set of) categories afterwards. For protocol, we compare the number in the text against the protocol-specific values 104 or 2404. Each extracted claim is labeled as supported, unsupported, or contradicted with respect to the given bound. We consider an explanation to violate the bound if it contains at least one unsupported or contradicted claim.

Violation-type classification. We use a hybrid approach: a conservative violation-type lexicon augmented with spaCy POS tags and dependency-parsing features. Negations are not supported; we filter out negated violation sentences before classification. We also filter out sentences that contain meta-references to violation-type terms without asserting a violation (e.g., “This is not a protocol violation.”), as they can be highly misleading. If spaCy is not available, REASONGUARD falls back to a simplified, substring-based implementation without NLP, mirroring the basic RV approach.

Ground truth. We collect labels on a sample of 200 (bound, explanation) pairs. To avoid confirmation bias, the annotator does not see REASONGUARD’s violation labels. Our goal was a random sample of violations across categories; however, given the small sample size and interest in rare categories such as hallucinated types, we oversample violations and underrepresent clean samples.

In a pilot annotation study, a single annotator labeled all samples in succession. This marathon annotation study was very noisy, with $\kappa \approx 0$ against REASONGUARD’s labels, likely due to annotator fatigue. We discard those labels and restart with breaks every 25 samples. For a second-rater check, we sent a separate 50 samples to a domain expert at Brno University of Technology.

Suite overview. In total, we evaluate 1,000 responses. Half are synthetic, hand-constructed responses used only for regression-testing REASONGUARD itself; these responses intentionally exhibit all four violations (V1–V4) alongside clean responses. The remaining responses come from our IEC dataset: 300 responses from the three local models and 200 responses from the two cloud-based models. We use the regression-test results to verify that REASONGUARD works as expected, but do not count them toward model-level conclusions.

Metrics and benchmarks. We report class-level precision, recall, and F1, as well as accuracy. We report macro-F1 (the average of per-class F1) as an aggregated metric and use Cohen’s κ as an agreement metric. We pre-register the benchmarks $\kappa \geq 0.70$ and macro-F1 ≥ 0.80 before analysis. Associations among violation type, model, and event type are analyzed with chi-square tests of independence.

Design rationale. REASONGUARD is designed to be deterministic: given the same explanation and bound, the set of violations is the same each time REASONGUARD checks the explanation. Consequently, REASONGUARD favors precision over recall for open-set claims. REASONGUARD is designed to provide deployers of AI components in critical infrastructure a list of violations they can act upon, and we argue that discrete violation types are more actionable than faithfulness scores.

V. RESULTS

We now discuss the results of this study. First, we analyze the distribution of violations in the LLM’s responses. Then, we analyze the results from REASONGUARD, our faithfulness verifier, comparing them against a smaller set of human-annotated data. We perform chi-square tests of independence

TABLE II

PER-MODEL OUTCOME DISTRIBUTION (SINGLE LABEL PER RESPONSE; $n=100$ PER REAL MODEL, 500 SYNTHETIC). CLEAN = NO VIOLATION.

Model	Clean	V1	V4	V5	Mixed
llama3.1:8b	63	17	15	0	5
mistral:7b	33	52	0	9	6
gpt-4o-mini	28	72	0	0	0
phi3:mini	2	58	2	1	37
gemini-2.5-flash-lite	0	0	100	0	0
synthetic (regression)	100	0	125	0	275

to understand the relationship between event types and violation types. Finally, we discuss the latency of our approach.

A. Distribution of violation types

Table II shows the distribution of violation types for each of the tested language models. In general, we observe an order-of-magnitude difference between models with the most (llama3.1:8b which had 63%) and least (gemini-2.5-flash-lite which had 0%) clean (NONE) responses. In general, larger local models performed better, but alignment and decoding strategies had a big impact on the cloud-based LLMs. Some qualitative differences between models include gpt-4o-mini violating constraints in the form of high-severity hallucination (V1) the most, gemini-2.5-flash-lite never hallucinating but always under-specifying (V4), and phi3:mini hallucinating and underspecifying often and producing incoherent responses frequently. Overall, given sufficiently stringent prompts, LLMs are more likely to hallucinate facts (V1) rather than reversing them (V2), and to misspecify by omission (V4) rather than overgeneralization (V3).

B. Hallucination detection

Table III summarizes the agreement between REASONGUARD’s violation type predictions and the 200 human-provided labels. Overall, the agreement is poor and does not meet our preregistered performance goals: the detector’s accuracy is 0.30, macro-F1 score is 0.19, and Cohen’s κ is 0.16. However, looking at individual violation types, we observe a more fine-grained picture. For hallucination (V1), the detector has high precision (0.90) but low recall (0.09). The detector manages to pick up on 9 out of the 98 human-annotated V1 violations. The remaining violations were classified as NONE, MIXED, or underspecification (V4). For underspecification (V4), REASONGUARD has middling precision (0.48) but high recall (0.92): it flags 23 of the 25 V4 violations annotated by the human rater, but also overpredicts V4 frequently.

C. Chi-square tests

We conducted chi-square tests of independence (table IV) between violation types and producing LLMs, and violation types and event types, respectively. Both times, we rejected the null hypothesis (that violation type is independent of the other variable) at the 0.1% significance level. However, in the latter case, it appears that certain types of events are easier for certain models rather than there being a general trend

TABLE III

PER-CLASS DETECTOR PERFORMANCE ON THE 200-PAIR ANNOTATED SET (191 SINGLE-LABEL PAIRS SHOWN; 9 MULTI-LABEL MIXED OMITTED). OVERALL: ACCURACY 0.30, MACRO-F1 0.19, COHEN’S κ = 0.16.

Class	Precision	Recall	F1	Support
NONE	0.26	0.43	0.33	49
V1	0.90	0.09	0.17	98
V2	0.00	0.00	0.00	17
V3	0.00	0.00	0.00	2
V4	0.48	0.92	0.63	25
V5	—	—	—	0
Macro	0.27	0.24	0.19	191

TABLE IV

χ^2 TESTS OF INDEPENDENCE (VIOLATION CATEGORY VS. GROUPING VARIABLE). † FIVE REAL MODELS; EVENT-TYPE TEST EXCLUDES ZERO-COUNT CLASSES V2, V3.

Grouping variable	χ^2	df	p
Model† ($N=500$)	649.05	16	< .001
Proxy event type	39.09	12	< .001

that certain events are easier/harder across models. Therefore, the claim that “rare events cause more hallucination” is not accurate.

D. Latency

Our pipeline can be executed entirely on an Apple M-series CPU without GPU acceleration and responds in a timely fashion for interactive use. This allows REASONGUARD to be used offline, for example in situations where control rooms have limited or intermittent internet connectivity.

VI. DISCUSSION

A deployer evaluating LLMs for a safety-critical task may prefer gemini-2.5-flash-lite (0% clean) over gpt-4o-mini (28%) because it never hallucinates (it always fails, but only via under-specification, a low-severity failure)—even though this is the opposite of what they would be led to infer if presented with only a single, aggregate, scalar performance value. This example highlights how important it is to analyze LLM output for all relevant failure types, which is what we have striven to do in this paper.

A. Interpreting the low agreement

REASONGUARD’s agreement score (Cohen’s κ = 0.16) is lower than we would have liked; nevertheless, we hope this work and its limitations will prove useful to others. We can pinpoint the cause of REASONGUARD’s low κ as stemming from a single rule (V4), which was misaligned with our annotator. Whereas our annotator understood this rule to refer to caveats regarding uncertainty or using a proxy, REASONGUARD interpreted it as referring to all elements pertaining to the traffic and consequently fired it much more frequently, in about 55% of explanations, than did the human (12.5%). Rather than hyper-tune REASONGUARD’s rules until it agrees more closely with the annotator, thereby inflating its macro-F1 score, we have chosen to retain this rule and report REASONGUARD’s performance as-is.

B. REASONGUARD as a pre-filter

Though not on par with our annotator, REASONGUARD can still be used. Its poor κ reflects the impact of V4's over-triggering; its other rules perform well and its per-rule P/R scores are informative. For example, V1's high P (0.90) means REASONGUARD's fabrication detections provide a lower-bound on fabrications, despite V1's low R (0.09) indicating REASONGUARD misses many. Consequently, rather than a standalone, fully automated verifier, REASONGUARD could be deployed as a pre-filter for explanations in a human-in-the-loop validation workflow: any fabrication it detected can be assumed to be true and prioritized for inspection, saving human effort.

C. Mapping to the EU AI Act

REASONGUARD is not, by itself, sufficient to ensure conformity with the EU AI Act, but provides auditable, verifiable components that could be incorporated into an operator's quality-management system. By reporting per-class P/R rather than aggregate performance, it meets Article 13 transparency expectations. Runs are tracked in MLflow and artifacts, including this report, are versioned. This supports Article 17 monitoring and record-keeping expectations, especially when REASONGUARD is incorporated into a CI/CD pipeline, thereby supporting continuous monitoring via re-evaluation on update.

D. Threats to validity

REASONGUARD was verified against a bound, rather than the official NES@FIT automaton. Only one person annotated our 200 test-pairs; a second annotation is underway. We tested REASONGUARD using just one protocol (IEC-104), prompt-format and quantization (4-bit). Finally, each cell in our per-event-type sub-analysis accounts for 25 responses. While this was sufficient to identify systematic differences between cells, this small sample may limit statistical power to detect less-consistent patterns.

VII. FUTURE WORK

Immediate follow-ups include releasing a version of REASONGUARD that uses the official automaton schema in place of the current proxy variable, `formal_class`, allowing us to rerun the entire process in a deterministic fashion; learning a per-claim classifier on our released annotation dataset (200 explanation-verdict pairs), then comparing it against the rule-based, auditable version using an MLflow evaluation harness, which may increase recall on real violations (V1) while controlling false alarms (V4), and reporting both aggregate agreement and class-wise F1; expanding the number of protocols to include Modbus, DNP3, and S7 to assess generalizability, and identifying new violation categories, such as replay-attack. Ultimately, however, what matters is whether annotating explanations with V1–V5 violation classes leads to better decisions by human operators. To study this, we will conduct an in-the-loop experiment in which humans receive detector verdicts alongside explanations, either unlabeled or with violation classes attached. This will allow us to move

beyond agreement-based evaluation to measure the effect of REASONGUARD annotations on human decisions.

VIII. CONCLUSION

REASONGUARD is a runtime-verification system that uses a formally verified detector's verdict as a bound to judge LLMs' explanations, producing actionable, severity-mapped deviations in five classes. Our experiments with five LLMs on IEC-104 smart-grid data show statistically associated violation classes per model and per event, revealing model-specific failure modes missed by scalar metrics.

We openly report two limitations: our formal bound is a schema proxy pending the official automaton, and our rule-based verifier acts as an effective hallucination pre-filter but falls short as a standalone verifier ($\kappa = 0.16$), a gap traced to a single rule. Exposing such limitations is safer than false assurances in safety-critical settings; REASONGUARD surfaces them as explicit, auditable violations tied to EU AI Act requirements. All code, prompts, and data are publicly released [5].

ACKNOWLEDGMENT

I am grateful to Prof. Dr. Gayan de Silva (SRH University Hamburg) and Assoc. Prof. Dr. Ondřej Ryšavý and the NES@FIT group (Brno University of Technology) for providing datasets, documentation on the automata, and expert annotation.

REFERENCES

- [1] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On faithfulness and factuality in abstractive summarization," in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 1906–1919.
- [2] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FACTScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proc. 2023 Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [3] O. Honovich, R. Aharoni, J. Herzig, H. Taitelbaum, D. Kukliansky, V. Cohen, T. Scialom, I. Szpektor, A. Hassidim, and Y. Matias, "TRUE: Re-evaluating factual consistency evaluation," in *Proc. 2022 Conf. of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2022.
- [4] European Parliament and Council of the European Union, "Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act)," Official Journal of the European Union, 2024, <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>.
- [5] R. Bilalov, "ReasonGuard: Source code, evaluation scripts, and annotated dataset," Zenodo, <https://doi.org/10.5281/zenodo.22724731>, 2026.
- [6] E. M. Clarke, O. Grumberg, and D. A. Peled, *Model Checking*. MIT Press, 2001.
- [7] V. Havlena, P. Matoušek, O. Ryšavý, and L. Holík, "Accurate automata-based detection of cyber threats in smart grid communication," *IEEE Transactions on Smart Grid*, vol. 14, pp. 2352–2366, 2023.
- [8] M. Leucker and C. Schallhart, "A brief account of runtime verification," *Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009.
- [9] P. Matoušek, O. Ryšavý, and R. Grofčík, "ICS dataset for smart grid anomaly detection," 2022.
- [10] O. Ryšavý and P. Matoušek, "Modbus dataset for ICS anomaly detection," 2021.